

URL Shortener Learning Journal

Project Goal

We are building a real full-stack URL shortener instead of stopping at a CRUD API. The product will let a user create short links, visit them through redirects, review click analytics, edit destinations, and delete links from a simple browser dashboard.

Stack Choice

- Node.js is the runtime for our server.
- Express handles routing and middleware.
- MongoDB stores URL data.
- Mongoose gives us schemas and model methods.
- Zod validates incoming request bodies.
- Nano ID generates short, collision-resistant codes.

Folder Structure

```
url-shortener/  
  docs/  
  scripts/  
  src/  
    config/  
    controllers/  
    middleware/  
    models/  
    public/  
    routes/  
    services/  
    utils/
```

What We Did First

- Created a fresh project folder so this app does not interfere with the older Go task tracker in the workspace.
- Added a production-style folder structure instead of placing all logic in one file.
- Planned a separate learning artifact so the implementation and the explanation stay in sync.

Why This Matters

This first step teaches an important backend habit: before writing endpoint logic, decide how the codebase will be organized. Good structure makes everything easier later, especially validation, testing,

refactoring, and deployment.

Backend Architecture

- `src/app.js` is the entry point. It loads environment variables, connects to MongoDB, applies middleware, serves the frontend, and mounts routes.
- `src/models/Url.js` defines the database shape. Each document stores the original URL, a normalized URL, the short code, click count, and the last visit timestamp.
- `src/controllers/urlController.js` holds the request-response logic for the API and redirect flow.
- `src/services/urlService.js` contains reusable domain logic such as URL normalization and unique short code generation.
- `src/middleware/` contains shared HTTP behavior, including 404 handling and centralized error responses.

API Design Decisions

- `POST /api/urls` creates a short URL.
- `GET /api/urls` lists recent URLs for the dashboard.
- `GET /api/urls/:shortCode` returns analytics for one URL.
- `PATCH /api/urls/:shortCode` updates the destination URL.
- `DELETE /api/urls/:shortCode` removes a short URL.
- `GET /:shortCode` performs the real redirect.

This separation is important. API routes live under `/api`, while the redirect route lives at the root so the short links look clean.

Validation and Error Handling

- Zod validates request bodies before business logic runs.
- Invalid URLs are converted into a 400 Bad Request instead of a generic server error.
- Duplicate custom aliases return 409 Conflict.
- Missing records return 404 Not Found.
- Successful creation returns 201 Created.
- Successful deletion returns 204 No Content.

These status codes matter because they make the API predictable for frontend code, Postman testing, and future integrations.

Redirect and Analytics Logic

When someone opens a short URL:

1. The server looks up the shortCode.
2. If the code exists, it increments the click counter.
3. It updates lastVisitedAt.
4. It sends an HTTP redirect to the destination.

This is a good example of backend work that is not just CRUD. One request both changes data and produces browser behavior.

Frontend Product Layer

The app is usable without Postman because `src/public/` contains a small dashboard:

- a form for creating short links
- an optional custom alias field
- a live result box with copy support
- a recent-links view
- edit and delete actions

This is useful to learn because backend projects become much easier to demo when they include a thin but polished UI.

Reliability Improvements Added

- Duplicate long URLs reuse the existing short link unless the user asks for a custom alias.
- Short code generation checks for collisions before saving.
- Centralized error handling keeps controller code cleaner.
- The app uses `BASE_URL` so responses can return a complete short URL string.

Local Setup Notes

The code is ready, but this machine does not currently have MongoDB installed. To run the product end to end, use one of these:

- install local MongoDB and keep `MONGO_URI=mongodb://127.0.0.1:27017/url-shortener`
- use MongoDB Atlas and replace `MONGO_URI` in `.env`

For this project, we switched to MongoDB Atlas so the app can run without installing a local database server. That is a very common real-world setup for portfolio apps and small production projects.

Verification So Far

- Installed all Node dependencies successfully.
- Loaded the main backend modules with Node to catch import/runtime issues early.
- Ran syntax checks on every JavaScript file.
- Generated a PDF snapshot of this learning journal.

Next Verification Step

Now that the Atlas connection string is available, the next step is full end-to-end testing:

- start the Express server
- confirm the MongoDB connection succeeds
- create a short URL through the API or UI
- verify the redirect works
- verify click analytics update after the redirect

What To Learn From This Stage

- Controllers should stay focused on HTTP behavior.
- Reusable business logic belongs in services and utilities.
- Validation should happen before database writes.
- Real products need a user-facing layer, not only raw endpoints.
- A project becomes portfolio-ready when the code, docs, and demo path all work together.

Bug Caught During Live Testing

The first live Atlas test exposed a startup bug:

- the app called `connectDB()` without waiting for it
- Express started listening immediately
- the process could print "Server running" even though MongoDB had not connected yet

We fixed that by wrapping startup in an async `startServer()` function and awaiting the database connection before calling `app.listen()`.

This is an important backend lesson: startup order matters. If your app depends on the database, treat database connectivity as part of bootstrapping, not background work.